

Javascript

Intro, data types, variables, scope, unary, arithmetic

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Javascript</title>
    <!-- <script></script> -->
  </head>
  <body>
    <h1>Javascript</h1>

    <!-- javascript -> scripting language -->

    <!-- programming langugae vs scripting language -->

    <!-- dynamic typed language vs statically typed language -->

    <!-- 1995 -> Brendan Eich -> livescript -> javascript -->

    <script>
      // how does the javascript runs -> browser has javascript engine
      // ( intrepeter language)

      // chrome javascript engine - v8
      // firefox -> spider monkey
      console.log("Hello there");

      // data types and variables

      console.log(typeof 10);

      console.log(typeof "Fiit");

      console.log(typeof true);

      console.log(typeof []);

      console.log(typeof {});

      console.log(typeof undefined);

      console.log(typeof null);

      // data types -> primitve and non-primitive
```

```

// primitive data types -> fixed value

// non-primitive data types -> dynamic value or non fixed value

// array , object

// javascript keyword -> var let , const

var a = 10;

//declaration
var b;
var b; // var keyword allows re-declare

//definition
b = 20;
b = 30; // allows re-inizliation
console.log(b);
console.log(a);

let fiit;
// let fiit; // let keyword don't allow re-declare

fiit = 10;

fiit = 100; // allow re-initilization

console.log(fiit);

const PI = 3.14;

// const PI = 3.15; // don't allow re-declare

// PI = 350; // don't allow re-initializaiton

// var -> redelcare, reinitizaion
// let -> no re-declare, re-initlization
// const -> don't allow both

// scope -> global scope, function scope, blocked scope
var data = 60; // global scope

function add() {
  var scope = 30; // function scope or global scope
  console.log(scope);
  console.log(data);
}

```

```

    add();

//    console.log(scope)

    let scoped = 50;
    function sub(){
        let fscoped = 60;
        console.log(fscoped);

        console.log(scoped);
    }

    sub();
//    console.log(fscoped);

{
    let y = 10;
    const z = 20;
    var x = 30;
}
// console.log(y);
// console.log(z);
console.log(x);

// operators -> unary, binary, ternary
// unary operator -> ++, --

let operand = 10;

console.log(operand++);
console.log(++operand);
console.log(operand--);
console.log(--operand);
console.log(operand);

// binary operators -> arithmetic

// +, -, *, /, %

let one = 10;
let two = 20;

console.log(one + two);

console.log(one - two);

```

```
console.log(one * two);

console.log(two / one );

console.log(7 % 4);

</script>
</body>
</html>
```

Theory Notes

1. JavaScript Basics

- **Invented in 1995** by Brendan Eich, originally called *LiveScript*.
- Runs inside browsers using **JavaScript engines**:
 - Chrome → V8
 - Firefox → SpiderMonkey

2. Data Types

- **Primitive** → string, number, boolean, null, undefined, symbol, bigint.
- **Non-primitive** → object, array, function.
- Strings can be defined with " ", ' ', or backticks ` ` (template literals).

3. Variables

- **var** → allows re-declare and re-initialize, function/global scope.
- **let** → no re-declare, allows re-initialize, block scope.
- **const** → no re-declare, no re-initialize, block scope.

4. Scope

- **Global scope** → accessible everywhere.
- **Function scope** → variables declared inside a function.
- **Block scope** → variables declared inside {} with let or const.

5. Operators

- **Unary** → ++, -- (increment/decrement).
- **Binary** → arithmetic (+, -, *, /, %).
- **Ternary** → condition ? trueValue : falseValue.

Coding Round Questions

1. Write a program to declare variables using var, let, and const. Show which ones allow re-declaration and re-initialization.
2. Demonstrate global scope, function scope, and block scope with examples.
3. Write a program to show the difference between operand++ and ++operand.
4. Perform arithmetic operations (+, -, *, /, %) on two numbers.
5. Print the type of different values: number, string, boolean, array, object, null, undefined.